

Competitive Programming

Section 67 Editorial: Persistence, Rollback, And Time

Competitive Programming Course

Table of contents

1	A. Branching Multiset	3
1.1	Model and algorithm	3
1.2	Correctness proof	3
1.3	Complexity and tests	3
1.4	Full C++ solution	3
1.5	Full Python solution	4
2	B. Versioned Range Add	7
2.1	Model and algorithm	7
2.2	Correctness proof	7
2.3	Complexity and tests	7
2.4	Full C++ solution	7
2.5	Full Python solution	9
3	C. Dynamic Connectivity	11
3.1	From operations to lifetimes	11
3.2	Correctness proof	11
3.3	Complexity and tests	11
3.4	Full C++ solution	11
3.5	Full Python solution	13
4	D. Tree Path K-th	16
4.1	Restatement and constraints	16
4.2	From path inclusion-exclusion to four roots	16
4.3	Pseudocode	16
4.4	Correctness proof	17
4.5	Complexity	17
4.6	Common wrong answers and tests	17
4.7	Full C++ solution	17
4.8	Full Python solution	20
5	Stress testing	23
6	What to carry forward	24

1 A. Branching Multiset

1.1 Model and algorithm

Use a persistent frequency segment tree on values $1..M$. A node stores the number of multiset elements in its interval. Insert and erase clone one root-to-leaf path and change counts by $+1$ or -1 ; the resulting root is the next version. A prefix query descends only into intervals intersecting $[1, x]$. For k -th, compare k with the left-child count and choose one child.

After inserting 5 into version zero and 2 into that result, only the paths to 5 and 2 were copied; both roots still share every other zero subtree.

1.2 Correctness proof

Induct on updates. The new leaf has the required changed multiplicity. Every copied ancestor sums its correct new child and its shared unchanged child, so the new root represents the requested version; older roots were never mutated. Prefix recursion partitions exactly the values at most x . At a k -th step, the left count is precisely the number of smaller candidates, so the chosen branch contains the requested rank. Thus every answer is correct.

1.3 Complexity and tests

Each operation is $O(\log M)$; memory is $O(\text{number_of_updates} \log M)$. Test branching, duplicate values, erasing the last copy, and minimum/maximum ranks.

1.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;
struct Node {
    int left = 0, right = 0, count = 0;
};
vector<Node> tree(1);
int update(int old, int lo, int hi, int position, int delta) {
    int node = tree.size();
    tree.push_back(tree[old]);
    tree[node].count += delta;
    if (lo != hi) {
        int mid = (lo + hi) / 2;
        if (position <= mid)
```

```

        tree[node].left = update(tree[old].left, lo, mid, position, delta);
    else
        tree[node].right = update(tree[old].right, mid + 1, hi, position, delta);
    }
    return node;
}

int prefix(int node, int lo, int hi, int x) {
    if (!node || x < lo) return 0;
    if (hi <= x) return tree[node].count;
    int mid = (lo + hi) / 2;
    return prefix(tree[node].left, lo, mid, x) + prefix(tree[node].right, mid + 1, hi, x);
}

int kth(int node, int lo, int hi, int k) {
    if (lo == hi) return lo;
    int mid = (lo + hi) / 2, left_count = tree[tree[node].left].count;
    if (k <= left_count) return kth(tree[node].left, lo, mid, k);
    return kth(tree[node].right, mid + 1, hi, k - left_count);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int maximum, q;
    if (!(cin >> maximum >> q)) return 0;
    vector<int> roots(1, 0);
    while (q--) {
        char type;
        int version, x;
        cin >> type >> version >> x;
        if (type == 'I' || type == 'E')
            roots.push_back(update(roots[version], 1, maximum, x, type == 'I' ? 1 : -1));
        else if (type == 'K')
            cout << kth(roots[version], 1, maximum, x) << '\n';
        else
            cout << prefix(roots[version], 1, maximum, x) << '\n';
    }
}

```

1.5 Full Python solution

```

import sys

def main() -> None:
    data = sys.stdin.buffer.read().split()
    if not data:
        return
    maximum, q = map(int, data[:2])
    index = 2

```

```

left = [0]
right = [0]
count = [0]
roots = [0]

def update(old, lo, hi, pos, delta):
    node = len(count)
    left.append(left[old])
    right.append(right[old])
    count.append(count[old] + delta)
    if lo != hi:
        mid = (lo + hi) // 2
        if pos <= mid:
            left[node] = update(left[old], lo, mid, pos, delta)
        else:
            right[node] = update(right[old], mid + 1, hi, pos, delta)
    return node

def prefix(node, lo, hi, x):
    if not node or x < lo:
        return 0
    if hi <= x:
        return count[node]
    mid = (lo + hi) // 2
    return prefix(left[node], lo, mid, x) + prefix(right[node], mid + 1, hi, x)

def kth(node, lo, hi, k):
    while lo != hi:
        mid = (lo + hi) // 2
        amount = count[left[node]]
        if k <= amount:
            node = left[node]
            hi = mid
        else:
            k -= amount
            node = right[node]
            lo = mid + 1
    return lo

output = []
for _ in range(q):
    kind = data[index].decode()
    version = int(data[index + 1])
    x = int(data[index + 2])
    index += 3
    if kind in "IE":
        roots.append(
            update(roots[version], 1, maximum, x, 1 if kind == "I" else -1)
        )
    elif kind == "K":
        output.append(str(kth(roots[version], 1, maximum, x)))

```

```
        else:
            output.append(str(prefix(roots[version], 1, maximum, x)))
        print("\n".join(output))

if __name__ == "__main__":
    main()
```

2 B. Versioned Range Add

2.1 Model and algorithm

Build version zero normally. Each persistent node stores `sum` and a local `lazy` addition applying to its complete interval. On full cover, clone the node and update both fields. On partial cover, clone only intersected paths and set

```
sum = left.sum + right.sum + lazy * interval_length.
```

No push is needed. During a query, accumulate ancestor lazy values. A fully covered node contributes `node.sum + carry*length`.

2.2 Correctness proof

The stored-sum equation is true after building. A full update adds the same amount to every position, so both its sum and tag change correctly. A partial update preserves the equation by recomputation, while shared children remain valid for their old versions. Induction proves every root's array. Query carry contains exactly tags of strict ancestors, while `node.sum` already contains the node's own tag, so every requested element is included once.

2.3 Complexity and tests

Build time and memory are $O(n)$. Updates and queries are $O(\log n)$; each update allocates $O(\log n)$ nodes. Test overlapping tags, negative additions, singletons, and updates branching from the same old root.

2.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;
using int64 = long long;
struct Node {
    int left = 0, right = 0;
    int64 sum = 0, lazy = 0;
};
vector<Node> tree(1);
int build(const vector<int64>& a, int lo, int hi) {
    int node = tree.size();
```

```

tree.push_back({});
if (lo == hi)
    tree[node].sum = a[lo];
else {
    int mid = (lo + hi) / 2;
    tree[node].left = build(a, lo, mid);
    tree[node].right = build(a, mid + 1, hi);
    tree[node].sum = tree[tree[node].left].sum + tree[tree[node].right].sum;
}
return node;
}

int update(int old, int lo, int hi, int ql, int qr, int64 value) {
    int node = tree.size();
    tree.push_back(tree[old]);
    if (ql <= lo && hi <= qr) {
        tree[node].sum += value * (hi - lo + 1);
        tree[node].lazy += value;
        return node;
    }
    int mid = (lo + hi) / 2;
    if (ql <= mid) tree[node].left = update(tree[old].left, lo, mid, ql, qr, value);
    if (qr > mid) tree[node].right = update(tree[old].right, mid + 1, hi, ql, qr, value);
    tree[node].sum = tree[tree[node].left].sum + tree[tree[node].right].sum +
        tree[node].lazy * (hi - lo + 1);
    return node;
}

int64 query(int node, int lo, int hi, int ql, int qr, int64 carry = 0) {
    if (ql <= lo && hi <= qr) return tree[node].sum + carry * (hi - lo + 1);
    carry += tree[node].lazy;
    int mid = (lo + hi) / 2, res = 0;
    if (ql <= mid) res += query(tree[node].left, lo, mid, ql, qr, carry);
    if (qr > mid) res += query(tree[node].right, mid + 1, hi, ql, qr, carry);
    return res;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    if (!(cin >> n >> q)) return 0;
    vector<int64> a(n + 1);
    for (int i = 1; i <= n; i++) cin >> a[i];
    vector<int> roots{build(a, 1, n)};
    while (q--) {
        char type;
        int v, l, r;
        cin >> type >> v >> l >> r;
        if (type == 'A') {
            int64 x;
            cin >> x;
            roots.push_back(update(roots[v], 1, n, l, r, x));
        } else

```

```

    cout << query(roots[v], 1, n, l, r) << '\n';
}
}

```

2.5 Full Python solution

```

import sys

def main() -> None:
    data = sys.stdin.buffer.read().split()
    if not data:
        return
    n, q = map(int, data[:2])
    idx = 2
    a = [0] + list(map(int, data[idx : idx + n]))
    idx += n
    left = [0]
    right = [0]
    total = [0]
    lazy = [0]

    def new(old=0):
        left.append(left[old])
        right.append(right[old])
        total.append(total[old])
        lazy.append(lazy[old])
        return len(total) - 1

    def build(lo, hi):
        node = new()
        if lo == hi:
            total[node] = a[lo]
        else:
            mid = (lo + hi) // 2
            left[node] = build(lo, mid)
            right[node] = build(mid + 1, hi)
            total[node] = total[left[node]] + total[right[node]]
        return node

    def update(old, lo, hi, ql, qr, value):
        node = new(old)
        if ql <= lo and hi <= qr:
            total[node] += value * (hi - lo + 1)
            lazy[node] += value
            return node
        mid = (lo + hi) // 2
        if ql <= mid:

```

```

        left[node] = update(left[old], lo, mid, ql, qr, value)
    if qr > mid:
        right[node] = update(right[old], mid + 1, hi, ql, qr, value)
    total[node] = (
        total[left[node]] + total[right[node]] + lazy[node] * (hi - lo + 1)
    )
    return node

def query(node, lo, hi, ql, qr, carry=0):
    if ql <= lo and hi <= qr:
        return total[node] + carry * (hi - lo + 1)
    carry += lazy[node]
    mid = (lo + hi) // 2
    answer = 0
    if ql <= mid:
        answer += query(left[node], lo, mid, ql, qr, carry)
    if qr > mid:
        answer += query(right[node], mid + 1, hi, ql, qr, carry)
    return answer

roots = [build(1, n)]
out = []
for _ in range(q):
    kind = data[idx].decode()
    v, l, r = map(int, data[idx + 1 : idx + 4])
    idx += 4
    if kind == "A":
        value = int(data[idx])
        idx += 1
        roots.append(update(roots[v], 1, n, l, r, value))
    else:
        out.append(str(query(roots[v], 1, n, l, r)))
print("\n".join(out))

if __name__ == "__main__":
    main()

```

3 C. Dynamic Connectivity

3.1 From operations to lifetimes

Match every addition with its removal. The edge is active on `[add_time, remove_time)`; an unremoved edge ends at `q`. Store each lifetime in a segment tree over operation indices.

Traverse that tree with rollback DSU. Before entering a node, save the history length and unite all its edges. At a leaf, answer connectivity. On exit, restore the snapshot.

3.2 Correctness proof

For time `t`, an edge lifetime containing `t` is partitioned into time-tree nodes, exactly one of which lies on the root-to-leaf path for `t`. No lifetime excluding `t` has such a node. Therefore the DSU at leaf `t` contains exactly the active graph's edges. DSU unions preserve connected components, so the query is correct. Rollback restores the precise entry state, making sibling time intervals independent.

3.3 Complexity and tests

Each edge lifetime is stored in $O(\log q)$ nodes. Union/find costs $O(\log n)$ without path compression, so total time is $O(q \log q \log n)$ and memory is $O(q \log q + n)$. Test nested lifetimes, repeated lifetimes of the same edge, queries at removal time, and edges surviving to the end.

3.4 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;
struct DSU {
    vector<int> parent, size;
    vector<pair<int, int>> history;
    DSU(int n) : parent(n), size(n, 1) { iota(parent.begin(), parent.end(), 0); }
    int find(int x) {
        while (x != parent[x]) x = parent[x];
        return x;
    }
    int snapshot() { return history.size(); }
    void join(int a, int b) {
        a = find(a);
        b = find(b);
```

```

    if (a == b) return;
    if (size[a] < size[b]) swap(a, b);
    history.push_back({b, size[a]});
    parent[b] = a;
    size[a] += size[b];
}
void rollback(int snap) {
    while ((int)history.size() > snap) {
        auto [b, old] = history.back();
        history.pop_back();
        int a = parent[b];
        size[a] = old;
        parent[b] = b;
    }
}
};
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    if (!(cin >> n >> q)) return 0;
    struct Op {
        char type;
        int u, v;
    };
    vector<Op> ops(q);
    map<pair<int, int>, int> opened;
    vector<vector<pair<int, int>>> seg(4 * q + 4);
    function<void(int, int, int, int, int, pair<int, int>>> add =
        [&](int node, int lo, int hi, int ql, int qr, pair<int, int> e) {
            if (ql >= hi || qr <= lo) return;
            if (ql <= lo && hi <= qr) {
                seg[node].push_back(e);
                return;
            }
            int mid = (lo + hi) / 2;
            add(node * 2, lo, mid, ql, qr, e);
            add(node * 2 + 1, mid, hi, ql, qr, e);
        });
    for (int t = 0; t < q; t++) {
        cin >> ops[t].type >> ops[t].u >> ops[t].v;
        --ops[t].u;
        --ops[t].v;
        if (ops[t].u > ops[t].v) swap(ops[t].u, ops[t].v);
        auto e = pair{ops[t].u, ops[t].v};
        if (ops[t].type == '+')
            opened[e] = t;
        else if (ops[t].type == '-') {
            add(1, 0, q, opened[e], t, e);
            opened.erase(e);
        }
    }
}

```

```

}
for (auto [e, start] : opened) add(1, 0, q, start, q, e);
DSU dsu(n);
function<void(int, int, int)> dfs = [&](int node, int lo, int hi) {
    int snap = dsu.snapshot();
    for (auto [u, v] : seg[node]) dsu.join(u, v);
    if (hi - lo == 1) {
        if (ops[lo].type == '?')
            cout << (dsu.find(ops[lo].u) == dsu.find(ops[lo].v) ? "YES\n" : "NO\n");
    } else {
        int mid = (lo + hi) / 2;
        dfs(node * 2, lo, mid);
        dfs(node * 2 + 1, mid, hi);
    }
    dsu.rollback(snap);
};
if (q) dfs(1, 0, q);
}

```

3.5 Full Python solution

```

import sys

def main() -> None:
    tokens = sys.stdin.buffer.read().split()
    if not tokens:
        return
    n, q = map(int, tokens[:2])
    idx = 2
    ops = []
    opened = {}
    seg = [[] for _ in range(4 * q + 4)]

    def add(node, lo, hi, ql, qr, edge):
        if ql >= hi or qr <= lo:
            return
        if ql <= lo and hi <= qr:
            seg[node].append(edge)
            return
        mid = (lo + hi) // 2
        add(node * 2, lo, mid, ql, qr, edge)
        add(node * 2 + 1, mid, hi, ql, qr, edge)

    for time in range(q):
        kind = tokens[idx].decode()
        u = int(tokens[idx + 1]) - 1
        v = int(tokens[idx + 2]) - 1

```

```

idx += 3
if u > v:
    u, v = v, u
ops.append((kind, u, v))
edge = (u, v)
if kind == "+":
    opened[edge] = time
elif kind == "-":
    add(1, 0, q, opened.pop(edge), time, edge)
for edge, start in opened.items():
    add(1, 0, q, start, q, edge)
parent = list(range(n))
size = [1] * n
history = []

def find(x):
    while x != parent[x]:
        x = parent[x]
    return x

def join(a, b):
    a = find(a)
    b = find(b)
    if a == b:
        return
    if size[a] < size[b]:
        a, b = b, a
    history.append((b, size[a]))
    parent[b] = a
    size[a] += size[b]

def rollback(snapshot):
    while len(history) > snapshot:
        b, old = history.pop()
        a = parent[b]
        size[a] = old
        parent[b] = b

output = []

def dfs(node, lo, hi):
    snapshot = len(history)
    for u, v in seg[node]:
        join(u, v)
    if hi - lo == 1:
        kind, u, v = ops[lo]
        if kind == "?":
            output.append("YES" if find(u) == find(v) else "NO")
    else:
        mid = (lo + hi) // 2
        dfs(node * 2, lo, mid)

```

```
        dfs(node * 2 + 1, mid, hi)
        rollback(snapshot)

    if q:
        dfs(1, 0, q)
    print("\n".join(output))

if __name__ == "__main__":
    main()
```

4 D. Tree Path K-th

4.1 Restatement and constraints

Each query asks for an order statistic of the multiset of values on one tree path. Reconstructing the path and sorting it is linear or worse per query, which cannot handle $n, q \leq 200000$. Values require coordinate compression, and duplicates must contribute their full multiplicity.

4.2 From path inclusion-exclusion to four roots

Root the tree at vertex 1. For every vertex u , build a persistent frequency tree $\text{root}[u]$ containing the values on the root-to- u path, including both endpoints. It is obtained from its parent's root by inserting $\text{value}[u]$.

Let $w = \text{lca}(u, v)$ and let p be the parent of w ; use the empty root when w is the global root. For every compressed value interval, its multiplicity on path $u \dots v$ is

```
count(root[u]) + count(root[v])  
- count(root[w]) - count(root[p]).
```

The two positive prefixes contain the root-to- w prefix twice. Subtracting $\text{root}[w]$ and $\text{root}[\text{parent}(w)]$ removes everything above w twice but leaves the value at w once.

For the sample query from 4 to 3, the path values are 1,3,8,8. The four-root expression gives exactly these four multiplicities, so descending left when the combined left count is at least k returns 3 for $k=2$.

4.3 Pseudocode

```
compress all vertex values  
root[global_root] = insert(empty, value[global_root])  
for every later vertex u in parent-before-child order:  
    root[u] = insert(root[parent[u]], value[u])  
  
for query (u,v,k):  
    w = LCA(u,v)  
    p = parent(w), or the empty root  
    descend four segment trees together:  
        left_count = left[u] + left[v] - left[w] - left[p]  
        choose the left or right half as in ordinary k-th selection
```

4.4 Correctness proof

Lemma 1. $\text{root}[u]$ stores exactly the multiset of values on the rooted path from vertex 1 through u .

Proof. The root vertex inserts its value into the empty multiset. Every other vertex copies its parent's correct root and inserts its own value. Induction in parent-before-child order proves the claim. Old roots are never mutated. QED.

Lemma 2. The signed combination of the four roots gives exactly the value multiset on path $u..v$.

Proof. Both positive rooted paths share precisely the path from vertex 1 through w . Subtracting the prefix through w once and the prefix through $\text{parent}(w)$ once cancels every strict ancestor twice and cancels w only once. All vertices below w occur in exactly one positive prefix. QED.

Lemma 3. The simultaneous segment-tree descent returns the k -th smallest path value.

Proof. At every node, the four-root combination gives the exact number of path values in its left half by Lemma 2. If k does not exceed this count, the answer lies there. Otherwise all those values precede the answer, so subtract their count and enter the right half. The invariant continues until one compressed value remains. QED.

Together, the lemmas prove that every reported answer is the requested order statistic.

4.5 Complexity

Coordinate compression costs $O(n \log n)$. Building all roots uses $O(n \log n)$ time and nodes. LCA preprocessing uses $O(n \log n)$ memory and time. Each query takes $O(\log n)$ for LCA and $O(\log n)$ for the four-root descent.

4.6 Common wrong answers and tests

- Subtracting $\text{root}[w]$ twice, which removes the LCA value completely.
- Subtracting only $\text{root}[\text{parent}(w)]$, which leaves the shared prefix twice.
- Building one root per DFS time instead of one root per root-to-vertex path.
- Deduplicating values without retaining frequencies.
- Forgetting the empty-root case when the LCA is vertex 1.

Test $u=v$, a path whose values are all equal, an LCA equal to one endpoint, the global root as LCA, a chain, and a star.

4.7 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

struct Node {
    int left = 0;
```

```

    int right = 0;
    int count = 0;
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    cin >> n >> q;
    vector<int> value(n), compressed;
    for (int& x : value) cin >> x;
    compressed = value;
    sort(compressed.begin(), compressed.end());
    compressed.erase(unique(compressed.begin(), compressed.end()),
                      compressed.end());

    vector<vector<int>> graph(n);
    for (int edge = 1; edge < n; ++edge) {
        int u, v;
        cin >> u >> v;
        --u;
        --v;
        graph[u].push_back(v);
        graph[v].push_back(u);
    }

    int levels = bit_width(static_cast<unsigned>(n));
    vector<vector<int>> up(levels, vector<int>(n));
    vector<int> depth(n), order = {0};
    for (int index = 0; index < n; ++index) {
        int u = order[index];
        for (int v : graph[u]) {
            if (v == up[0][u] || v == 0) continue;
            up[0][v] = u;
            depth[v] = depth[u] + 1;
            order.push_back(v);
        }
    }

    for (int level = 1; level < levels; ++level) {
        for (int u = 0; u < n; ++u) {
            up[level][u] = up[level - 1][up[level - 1][u]];
        }
    }

    vector<Node> tree(1);
    auto insert = [&](auto&& self, int old, int low, int high,
                    int position) -> int {
        int current = static_cast<int>(tree.size());
        tree.push_back(tree[old]);
        ++tree[current].count;
    };

```

```

    if (low != high) {
        int middle = (low + high) / 2;
        if (position <= middle) {
            tree[current].left = self(self, tree[old].left, low, middle,
                                     position);
        } else {
            tree[current].right = self(self, tree[old].right, middle + 1,
                                       high, position);
        }
    }
    return current;
};

vector<int> root(n);
for (int u : order) {
    int rank = lower_bound(compressed.begin(), compressed.end(), value[u])
               - compressed.begin();
    int parent_root = (u == 0 ? 0 : root[up[0][u]]);
    root[u] = insert(insert, parent_root, 0,
                     static_cast<int>(compressed.size()) - 1, rank);
}

auto lca = [&](int u, int v) {
    if (depth[u] < depth[v]) swap(u, v);
    int difference = depth[u] - depth[v];
    for (int level = 0; level < levels; ++level) {
        if (difference >> level & 1) u = up[level][u];
    }
    if (u == v) return u;
    for (int level = levels - 1; level >= 0; --level) {
        if (up[level][u] != up[level][v]) {
            u = up[level][u];
            v = up[level][v];
        }
    }
    return up[0][u];
};

auto kth = [&](int a, int b, int c, int d, int k) {
    int low = 0;
    int high = static_cast<int>(compressed.size()) - 1;
    while (low != high) {
        int left_count = tree[tree[a].left].count
                        + tree[tree[b].left].count
                        - tree[tree[c].left].count
                        - tree[tree[d].left].count;
        int middle = (low + high) / 2;
        if (k <= left_count) {
            a = tree[a].left;
            b = tree[b].left;
            c = tree[c].left;

```

```

        d = tree[d].left;
        high = middle;
    } else {
        k -= left_count;
        a = tree[a].right;
        b = tree[b].right;
        c = tree[c].right;
        d = tree[d].right;
        low = middle + 1;
    }
}
return low;
};

while (q--) {
    int u, v, k;
    cin >> u >> v >> k;
    --u;
    --v;
    int ancestor = lca(u, v);
    int before = (ancestor == 0 ? 0 : root[up[0][ancestor]]);
    int rank = kth(root[u], root[v], root[ancestor], before, k);
    cout << compressed[rank] << '\n';
}
}

```

4.8 Full Python solution

```

import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    iterator = iter(data)
    n = next(iterator)
    q = next(iterator)
    values = [next(iterator) for _ in range(n)]
    coordinates = sorted(set(values))
    rank = {value: index for index, value in enumerate(coordinates)}

    graph = [[] for _ in range(n)]
    for _ in range(n - 1):
        u = next(iterator) - 1
        v = next(iterator) - 1
        graph[u].append(v)
        graph[v].append(u)

```

```

levels = n.bit_length()
up = [[0] * n for _ in range(levels)]
depth = [0] * n
order = [0]
for u in order:
    for v in graph[u]:
        if v == up[0][u] or v == 0:
            continue
        up[0][v] = u
        depth[v] = depth[u] + 1
        order.append(v)
for level in range(1, levels):
    previous = up[level - 1]
    current = up[level]
    for u in range(n):
        current[u] = previous[previous[u]]

left = [0]
right = [0]
count = [0]

def insert(old: int, low: int, high: int, position: int) -> int:
    current = len(count)
    left.append(left[old])
    right.append(right[old])
    count.append(count[old] + 1)
    if low != high:
        middle = (low + high) // 2
        if position <= middle:
            left[current] = insert(left[old], low, middle, position)
        else:
            right[current] = insert(right[old], middle + 1, high, position)
    return current

roots = [0] * n
maximum = len(coordinates) - 1
for u in order:
    parent_root = 0 if u == 0 else roots[up[0][u]]
    roots[u] = insert(parent_root, 0, maximum, rank[values[u]])

def lca(u: int, v: int) -> int:
    if depth[u] < depth[v]:
        u, v = v, u
    difference = depth[u] - depth[v]
    for level in range(levels):
        if difference >> level & 1:
            u = up[level][u]
    if u == v:
        return u
    for level in range(levels - 1, -1, -1):

```

```

        if up[level][u] != up[level][v]:
            u = up[level][u]
            v = up[level][v]
    return up[0][u]

def kth(a: int, b: int, c: int, d: int, k: int) -> int:
    low = 0
    high = maximum
    while low != high:
        left_count = (
            count[left[a]] + count[left[b]]
            - count[left[c]] - count[left[d]]
        )
        middle = (low + high) // 2
        if k <= left_count:
            a, b, c, d = left[a], left[b], left[c], left[d]
            high = middle
        else:
            k -= left_count
            a, b, c, d = right[a], right[b], right[c], right[d]
            low = middle + 1
    return low

answer = []
for _ in range(q):
    u = next(iterator) - 1
    v = next(iterator) - 1
    k = next(iterator)
    ancestor = lca(u, v)
    before = 0 if ancestor == 0 else roots[up[0][ancestor]]
    answer.append(str(coordinates[kth(
        roots[u], roots[v], roots[ancestor], before, k
    )]))
print("\n".join(answer))

if __name__ == "__main__":
    main()

```

5 Stress testing

The randomized generators copy full multisets and arrays for A/B, rebuild a fresh adjacency list plus BFS for every C query, and explicitly reconstruct and sort every D query path. These deliberately share no state-saving logic with the references.

6 What to carry forward

Persistence pays memory to preserve arbitrary branches. Rollback pays offline reordering to reuse one mutable state. Choose from the query timeline, not from the data structure's name.